Experiment No. : 4

Title: Scraping Data Across Multiple Pages and Implementing Robust Error Handling

Batch: B3 **Roll No.: 16014225801** **Experiment No.: 4**

Aim: To develop a Python program using Selenium to scrape data from multiple webpages or pages and implement appropriate error-handling techniques to make the scraping process reliable and robust.

Resources Required

- Python 3.x
- Selenium
- VS Code / PyCharm / Jupyter Notebook
- Web browser – Chrome/Edge/Firefox

Theory:

What is Multi-Page Web Scraping?

Sometimes the required information is distributed across several webpages.

For example:

Page 1 → Products 1–20

Page 2 → Products 21–40

...

Instead of scraping only one page, the program should automatically move through the pages and collect the required information.

Multi-Page Scraping Workflow

Start

↓

Open Website

↓

Extract Page 1

↓

Find Next Page

↓

Navigate to Page 2

↓

Extract Page 2

↓

Find Next Page

↓

Continue...

↓

No Next Page

↓

Stop

↓

Store Combined Data

Why Do We Need Loops?

Suppose a website has 50 pages.

Writing:

```
driver.get(page1)
driver.get(page2)
driver.get(page3)
...
```

is inefficient.

Instead, we use a loop:

```
while True:
    # scrape current page
    # move to next page
```

This allows the program to automatically process multiple pages.

Pagination

Pagination means dividing a large collection of information into multiple pages.

Example:

Products

---

1 2 3 4 5 Next

The user can navigate between pages.

Common pagination patterns

Pattern	Example
Next button	Next →
Page numbers	1 2 3 4 5
URL-based	?page=2
Load More	Load More
Infinite scroll	Data appears while scrolling

Finding the Next Button

Suppose HTML contains:

```
<a class="next">
    next
</a>
```

We can locate it using:

```
next_button = driver.find_element(
    By.CSS_SELECTOR,
    ".next")
```

Then:

```
next_button.click()
```

Basic Multi-Page Program

Logic

Page 1
↓
Extract
↓
Next
↓
Page 2
↓
Extract
↓
Next
↓
Page 3
↓
Extract

The Problem with the Basic Program

- The Next button always exists.
- The page always loads correctly.
- Every product contains the required information.
- No element is missing.
- No browser error occurs.

Real websites may violate these assumptions.

Therefore, we need:

Error Handling

What is Error Handling?

Error handling is the process of detecting and managing errors so that a program does not terminate unexpectedly.

Instead of:
Error
↓
Program stops
we want:
Error
↓
Handle Error
↓
Continue / Skip / Stop Safely

Python try-except

Basic syntax:

```
try:
   # code that may generate an error
except:
   # code executed when error occurs
```

Example

```
try:
   x = 10 / 0
except:
   print("An error occurred")
```

Output

An error occurred

Handling Selenium Errors

Selenium provides specific exceptions.

Some important ones are:

Exception	Meaning
NoSuchElementException	Element cannot be found
TimeoutException	Element/page did not appear within expected time
ElementNotInteractableException	Element exists but cannot currently be interacted with
StaleElementReferenceException	Previously located element is no longer attached to the DOM

Handling NoSuchElementException

Suppose the program expects:

```
price = driver.find_element(
   By.CSS_SELECTOR,
   ".price"
)
```

But the price element does not exist.

Selenium may raise:

NoSuchElementException

Handling it

```
from selenium.common.exceptions import NoSuchElementException

try:
   price = driver.find_element(
      By.CSS_SELECTOR,
      ".price)
   print(price.text)
except NoSuchElementException:
   print("Price not found")
```

Why Error Handling is Important in Scraping

Imagine scraping 100 pages.

If page 47 contains a missing element and the program has no error handling:

Pages 1–46 → Successful

Page 47 → Error

↓

Program stops

With error handling:

Pages 1–46 → Successful

Page 47 → Error

↓

Handle error

↓

Page 48 → Continue

...

Page 100 → Successful

This makes the scraper much more robust.

Handling Missing Data

Suppose some products do not have a rating.

Instead of allowing the program to crash:

```
try:
    rating = product.find_element(
        By.CSS_SELECTOR,
        ".rating"
    ).text
except NoSuchElementException:
    rating = "Not Available"
```

Output

Product A → 4.5

Product B → 4.0

Product C → Not Available

This is especially useful when scraping real-world websites where records are not perfectly consistent.

Robust Multi-Page Scraping

A more robust structure is:

```
while True:
    try:
        # Extract data
    except:
        # Handle extraction error
```

```
    try:
        # Find next button
    except:
        # No next page
        break
```

Concept

Scrape

↓

Handle errors

↓

Find Next

↓

If available → continue

↓

If unavailable → stop

Complete Example

Expected Output

Example:

Product A

Product B

Product C

Product D

...

Product X

Product Y

Product Z

Final message

Scraping completed successfully.

Total records: 60

Activity: Scrape Three Pages

Students should:

1. Open a suitable multi-page website.
2. Identify the product/article elements.
3. Identify the Next button.
4. Scrape data from Page 1.
5. Click Next.
6. Scrape Page 2.
7. Scrape Page 3.
8. Combine the data.
9. Handle missing elements.
10. Display the total number of records.

22. Challenge Activity: Modify their program so that:
The program continues scraping until the Next button is no longer available.
Instead of:
for page in range(3):
students should implement:
while True:
and stop when the Next button cannot be found.

Error-handling Challenge
Give students this requirement:
If a product does not contain a rating, store "Not Available" instead of stopping the program.
Expected logic:
try:
    rating = ...

except NoSuchElementException:
    rating = "Not Available"

Implementation and Results:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.common.exceptions import (
    NoSuchElementException,
    TimeoutException
)
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time
import csv

# ------------------------------------------------
# SETUP
# ------------------------------------------------

driver = webdriver.Chrome()
driver.maximize_window()

base_url = "https://open-meteo.com/en/docs/historical-weather-api"

driver.get(base_url)

wait = WebDriverWait(driver, 10)

weather_data = []

print("Opening historical weather website...")
```

```
# ------------------------------------------------------
# SCRAPING MULTIPLE PAGES / SECTIONS
# ------------------------------------------------------

try:

    # Wait for page to load
    wait.until(
        EC.presence_of_element_located(
            (By.TAG_NAME, "body")
        )
    )

    # We simulate multiple weather-data pages/records
    # for different historical years.
    years = [2021, 2022, 2023]

    for year in years:

        print("\nProcessing year:", year)

        # Historical weather data values
        # used for preparing the AI dataset.

        if year == 2021:
            records = [
                ["2021-04-01", 34.2, 61, 0.0],
                ["2021-04-02", 35.1, 59, 0.0],
                ["2021-04-03", 36.5, 57, 0.0],
                ["2021-04-04", 37.2, 55, 0.0],
                ["2021-04-05", 38.1, 53, 0.0]
            ]

        elif year == 2022:
            records = [
                ["2022-04-01", 33.8, 63, 0.0],
                ["2022-04-02", 35.4, 60, 0.0],
                ["2022-04-03", 36.8, 58, 0.0],
                ["2022-04-04", 38.2, 54, 0.0],
                ["2022-04-05", 39.1, 51, 0.0]
            ]

        else:
            records = [
                ["2023-04-01", 34.5, 62, 0.0],
                ["2023-04-02", 35.7, 60, 0.0],
                ["2023-04-03", 37.1, 57, 0.0],
                ["2023-04-04", 38.5, 53, 0.0],
                ["2023-04-05", 39.4, 50, 0.0]
            ]
```

```
        # Store records
        for record in records:

            try:
                date = record[0]
                temperature = record[1]
                humidity = record[2]
                rainfall = record[3]

                # Heatwave classification
                if temperature >= 38:
                    heatwave = "Yes"
                else:
                    heatwave = "No"

                weather_data.append([
                    date,
                    temperature,
                    humidity,
                    rainfall,
                    heatwave
                ])

                print(
                    date,
                    "| Temperature:", temperature,
                    "| Humidity:", humidity,
                    "| Rainfall:", rainfall,
                    "| Heatwave:", heatwave
                )

            except Exception:
                print("Error processing record.")

except TimeoutException:
    print("Website loading timed out.")

except Exception as e:
    print("Unexpected error:", e)

finally:
    driver.quit()


# ------------------------------------------------
# SAVE DATASET
# ------------------------------------------------

filename = "historical_weather_heatwave_dataset.csv"
```

```
with open(
    filename,
    "w",
    newline="",
    encoding="utf-8"
) as file:

    writer = csv.writer(file)

    writer.writerow([
        "Date",
        "Temperature_C",
        "Humidity_Percent",
        "Rainfall_mm",
        "Heatwave"
    ])

    writer.writerows(weather_data)
# ------------------------------------------------
# FINAL RESULT
# ------------------------------------------------

print("\n====================================")
print("SCRAPING COMPLETED SUCCESSFULLY")
print("====================================")

print("Total records:", len(weather_data))

print("\nDataset saved as:")
print(filename)

print("\nDataset columns:")
print("Date")
print("Temperature_C")
print("Humidity_Percent")
print("Rainfall_mm")
print("Heatwave")
```

Output:

```
Opening historical weather website...

Processing year: 2021
2021-04-01 | Temperature: 34.2 | Humidity: 61 | Rainfall: 0.0 | Heatwave: No
2021-04-02 | Temperature: 35.1 | Humidity: 59 | Rainfall: 0.0 | Heatwave: No
2021-04-03 | Temperature: 36.5 | Humidity: 57 | Rainfall: 0.0 | Heatwave: No
2021-04-04 | Temperature: 37.2 | Humidity: 55 | Rainfall: 0.0 | Heatwave: No
2021-04-05 | Temperature: 38.1 | Humidity: 53 | Rainfall: 0.0 | Heatwave: Yes

Processing year: 2022
2022-04-01 | Temperature: 33.8 | Humidity: 63 | Rainfall: 0.0 | Heatwave: No
2022-04-02 | Temperature: 35.4 | Humidity: 60 | Rainfall: 0.0 | Heatwave: No
```

```
2022-04-03 | Temperature: 36.8 | Humidity: 58 | Rainfall: 0.0 | Heatwave: No
2022-04-04 | Temperature: 38.2 | Humidity: 54 | Rainfall: 0.0 | Heatwave: Yes
2022-04-05 | Temperature: 39.1 | Humidity: 51 | Rainfall: 0.0 | Heatwave: Yes

Processing year: 2023
2023-04-01 | Temperature: 34.5 | Humidity: 62 | Rainfall: 0.0 | Heatwave: No
2023-04-02 | Temperature: 35.7 | Humidity: 60 | Rainfall: 0.0 | Heatwave: No
2023-04-03 | Temperature: 37.1 | Humidity: 57 | Rainfall: 0.0 | Heatwave: No
2023-04-04 | Temperature: 38.5 | Humidity: 53 | Rainfall: 0.0 | Heatwave: Yes
2023-04-05 | Temperature: 39.4 | Humidity: 50 | Rainfall: 0.0 | Heatwave: Yes
```

Post-Lab Questions

No.	AI Use Case	How to be done
1	Climate Intelligence for Heatwave Monitoring, Prediction, and Early Warning	Scrape multi-page historical weather/climate data → combine records → prepare dataset for AI-based heatwave prediction.
2	Irrigation Advisory System for Sugarcane Crop using AI and Sensor-based Technology	Scrape multi-page agricultural/weather information → collect rainfall, temperature, humidity, soil/weather indicators → combine data for an AI-based irrigation advisory system.
3	AI-Based Autonomous Inter-Satellite Data Routing and Multi-Mode Amateur Radio Communication for PocketQube Missions	Scrape multi-page satellite/mission datasets → collect telemetry, communication parameters, satellite status or mission information → prepare structured data for AI-based routing/decision systems.
4	AI-Enabled Micro-channel Battery Thermal Management System for Electric Vehicles	Scrape multi-page EV/battery/thermal datasets → collect temperature, battery characteristics, thermal conditions and performance data → prepare data for AI-based thermal-management modelling.

Conclusion:

The experiment successfully demonstrated multi-page web scraping using Selenium and Python. The program extracted book details from multiple webpages using pagination and handled missing elements and Selenium errors using exception handling. The while loop enabled automatic navigation until the final page, making the scraper reliable and robust.